TECHNICAL PROJECT SUMMARY

Papeer

A conversational RAG assistant for reading, querying, and fact-checking academic papers.

An agentic Retrieval-Augmented Generation system built on LangGraph and FastAPI, with a streaming React interface. Papers arrive by file upload, URL, or ArXiv lookup; every question is routed by an LLM to paper retrieval, live web search, or a literature-based claim-verification pipeline — each session fully isolated in its own vector collection and conversation checkpoint.

STACK	CODEBASE	GRAPH	EVALUATION
LangGraph · FastAPI React 19 · Qdrant	~2,200 LOC 17 source files	7 nodes · 3 routes 2 agent tools	DeepEval · 5 metrics 10 synthetic goldens

Source: `/Users/shee_sayan/ResearchGPT`

Package `rag-papeer-project` v0.1.0 · Python 3.12

01 Executive Summary

Papeer lets a student or researcher drop a set of papers into an isolated workspace and then interrogate them in plain language. What separates it from a standard RAG chatbot is that retrieval is not assumed: an LLM router first decides *what kind of question this is*, and only then chooses between the vector store, the live web, or a dedicated fact-checking subgraph that hunts for newer work superseding a paper's claim.

What the system does

Ingests papers from four sources	PDF / TXT / Markdown upload, arbitrary web URLs, and ArXiv by ID or title. Everything is chunked (1000 chars, 200 overlap) and embedded into a per-session Qdrant collection.
Routes every query	A structured-output LLM classifies the message into `retrieve`, `verify_claim`, or `direct_answer` before any work is done.
Retrieves agentically	A tool-calling agent picks its own search query, its own `k`, and whether to hit the vector store or Tavily web search — then a second LLM grades whether what came back is actually relevant, triggering one query rewrite if it isn't.
Verifies claims against literature	Two parallel Tavily searches (general web + `site:arxiv.org`) feed a fact-checker prompt that returns a verdict plus up to three superseding papers with links.
Streams and persists	Tokens arrive over Server-Sent Events; full graph state lands in SQLite, so sessions survive a backend restart. A `/btw` side-channel answers off-topic questions without polluting that history.

Health of the project

Strong

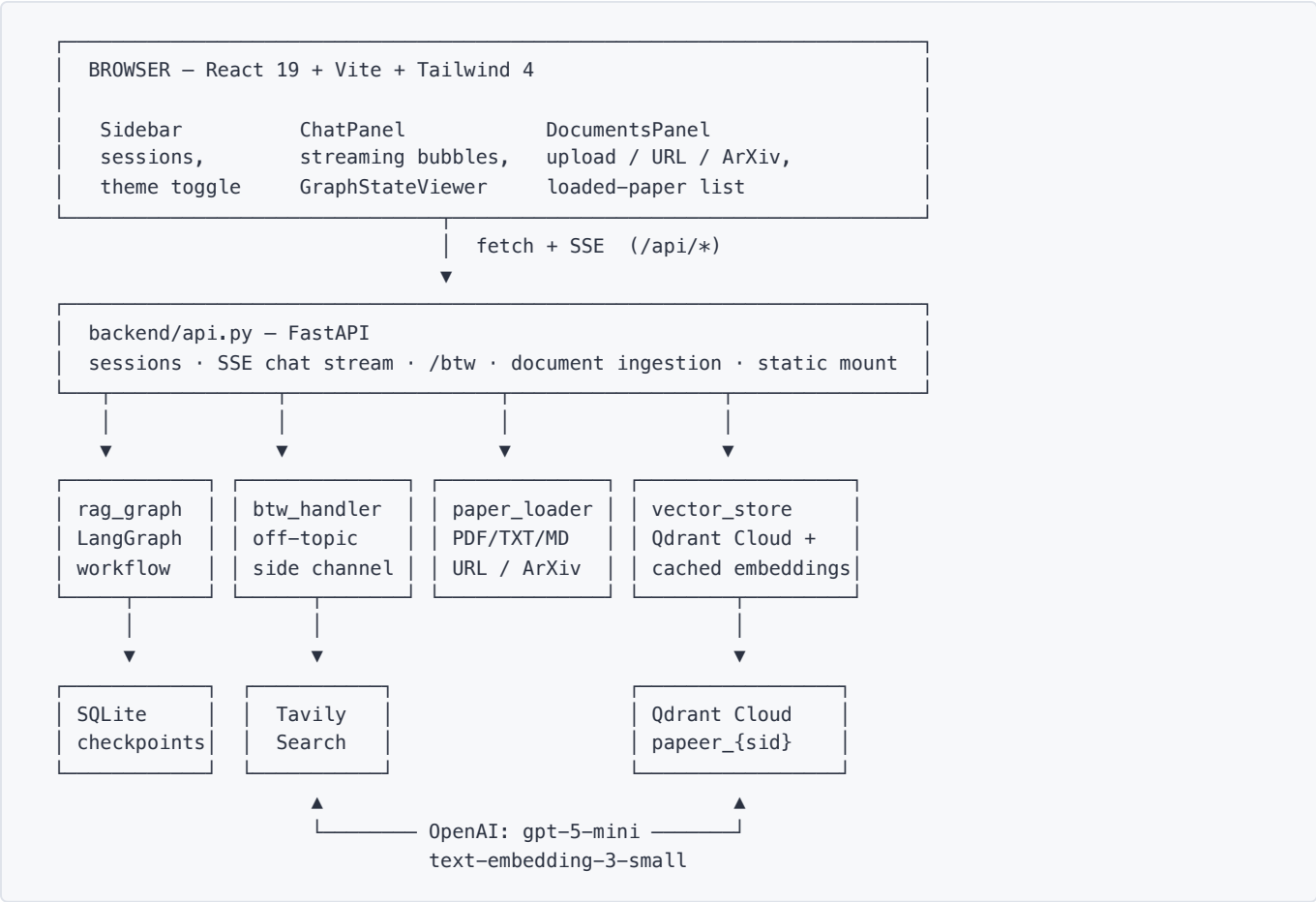
Clean module boundaries — the graph, the store, the loaders, and the API each own one concern. Faithfulness and answer relevancy score near-perfect (0.99), meaning the system essentially never hallucinates beyond its context. Deliberate production touches: embedding cache, singleton graph, temp-file cleanup, tool-call-ordering safeguards in the checkpointer.

Needs attention

Only 4 of 10 evaluation cases pass end-to-end, and the single bottleneck is *contextual relevancy* (avg 0.59) — the retriever pulls in too much unrelated text alongside the right answer. Three model-name inconsistencies and one documented-vs-actual rewrite-cap mismatch exist across the codebase. Container state is not volume-mounted.

02 System Architecture

Two deployable halves: a Vite/React SPA and a FastAPI process. In production the SPA is built to static assets that FastAPI itself mounts, collapsing the whole thing into a single container on port 8000.



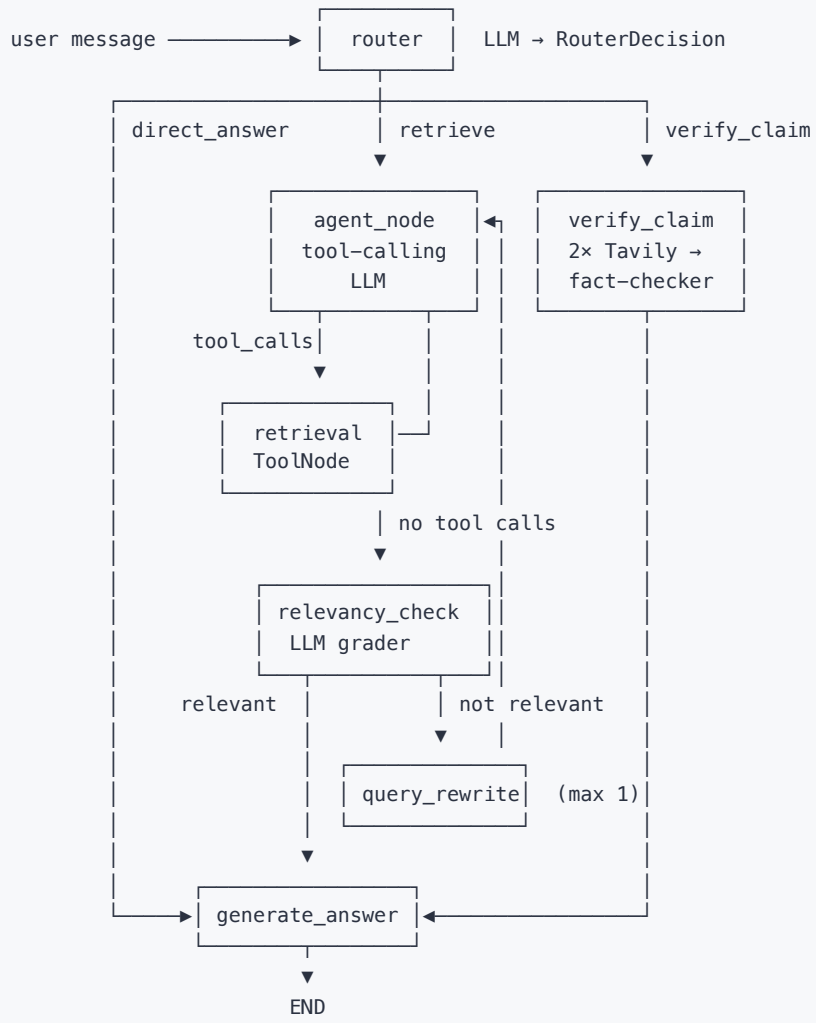
Module responsibilities

MODULE	LOC	RESPONSIBILITY
`backend/rag_graph.py`	435	The core. Defines `RAGState`, the router, two agent tools, the relevancy grader, the query rewriter, the claim verifier, the answer generator, and the graph wiring with a SQLite checkpointer.
`backend/api.py`	305	FastAPI surface. Session CRUD backed by `sessions.json` under a thread lock, SSE token streaming from `graph.stream()`, document upload endpoints, LLM-generated session titles, and a static mount for the built frontend.
`backend/paper_loader.py`	117	Source dispatch and chunking. PyMuPDF for PDFs, TextLoader for text/markdown, WebBaseLoader for URLs, and a hand-rolled ArXiv Atom API client (regex ID extraction → title search → PDF download) that replaced the unreliable `arxiv` package.
`backend/vector_store.py`	88	Qdrant Cloud client, per-session collection creation (1536-dim, cosine), and `CacheBackedEmbeddings` writing blake2b-keyed vectors to `./embedding_cache/`. Exposes `add_paper`, `list_papers`, `search`.
`backend/btw_handler.py`	46	Stateless generator for `/btw`. One structured call decides if web search is needed, then streams an answer. Never touches the vector store or the checkpointer.

`backend/models.py`	28	Five Pydantic schemas that constrain every structured LLM output: `RouterDecision`, `RelevancyDecision`, `BtwRouteDecision`, `SupersedingPaper`, `ClaimVerificationResult`.
`evaluate.py`	138	DeepEval harness. Synthesizes goldens from a reference PDF, runs each through a fresh session, scores five metrics with throttled concurrency, writes `eval_results.json`.
`frontend/src/`	681	Six components plus a typed SSE client. Per-session message maps, optimistic pending bubbles, incremental token append, and an expandable JSON graph-state viewer per turn.

03 The LangGraph Workflow

Seven nodes, three conditional branches, one cycle. Entry is always the router; exit is always `generate_answer`. The interesting design work is in the loop-control logic — two independent counters guard against two different runaway modes.



State schema

`RAGState` extends LangGraph's `MessagesState`, so the message list gets append-reducer semantics for free. Twelve fields travel through every node:

FIELD	TYPE	PURPOSE
`messages`	`list[BaseMessage]`	Inherited; the persisted conversation, appended not replaced
`session_id`	`str`	Injected into the retriever tool to scope the Qdrant collection
`query`	`str`	Current query — replaced in place when a rewrite fires
`route`	`str | None`	Router verdict; drives the first conditional edge
`retrieved_docs`	`list[Document]`	Accumulated context from both vector and web tools

`retrieval_attempts`	`int`	Tool-call counter, capped at `MAX_RETRIEVAL_ATTEMPTS = 3`
`rewrite_count`	`int`	Query-rewrite counter, capped at 1
`is_relevant`	`bool | None`	Grader verdict; decides rewrite vs. answer
`claim_verdict`	`str | None`	1–2 sentence fact-check summary
`claim_source`	`str | None`	URL of the primary superseding paper
`superseding_papers`	`list[dict] | None`	Up to three papers, each with title / url / summary
`answer`	`str | None`	Final text; also the non-streaming fallback the API reads

Node-by-node

router_node

A structured-output call returning `RouterDecision`, a `Literal` of three values so the LLM cannot invent a fourth. The prompt deliberately folds two very different cases into `retrieve`: questions about the uploaded papers, *and* questions whose answer changes over time (weather, prices, current events). The rationale is that both need fresh external context, and the downstream agent is the right place to decide which source. The prompt closes with an explicit tie-break — *"When in doubt between retrieve and direct_answer, prefer retrieve"* — biasing toward grounded answers over recalled ones.

agent_node + retrieval (ToolNode)

The agent is handed two tools and a system prompt that pushes decision-making down to it: it writes its own semantic query, chooses `k` between 1 and 10 based on how broad the question is, rewrites the query into keyword form for web search, and picks `max_results`. It is explicitly forbidden from answering — only from collecting. `parallel_tool_calls=False` keeps the loop to one tool per turn, making the message history linear and auditable.

Both tools return a `[ToolMessage, Command]` pair rather than a plain string. The `ToolMessage` is a short human-readable summary that goes into the LLM's context; the `Command` updates `retrieved_docs` in state. This keeps full document text out of the message history — the LLM sees *"Retrieved 4 chunk(s)"*, while the actual chunks travel in state and are only assembled into a prompt at answer time. That is a meaningful token saving on every agent turn.

> **DESIGN DETAIL WORTH NOTING**
>
> At the retrieval cap, `agent_node` swaps the tool-bound LLM for the plain one (`rag_graph.py:194`) instead of simply routing away. The comment explains why: cutting the graph off after the model has already emitted `tool_calls` would persist an `AIMessage` whose tool calls have no matching `ToolMessage`, permanently corrupting that session's checkpoint. `agent_routing` reinforces this by always executing pending tool calls before checking the cap. This is a bug class that only shows up once history is persisted, and it has been handled deliberately.

relevancy_check_node

Grades the first 300 characters of the top 3 retrieved chunks against the query. The prompt is tuned to be permissive — *"Be lenient: if there is any substantive overlap, return true"* — because a false negative costs a full rewrite-and-retrieve cycle while a false positive merely passes slightly weaker context to a generator that is already scoring 0.99 on faithfulness.

query_rewrite_node

Fires at most once. It rewrites the query toward more specific or domain-specific terminology, then resets `retrieved_docs`, `retrieval_attempts`, and `is_relevant` so the next pass starts clean, and pushes the rewritten text into `messages` as a new `HumanMessage` so the agent sees it as the live question.

verify_claim_node

Two Tavily calls at 5 results each — one phrased `"recent research superseding: {claim}"`, one scoped `site:arxiv.org`. Results are formatted into a labelled context block and passed to a fact-checker prompt with a hard constraint: *use only titles and URLs that appear verbatim in the provided results*. That single instruction is what keeps the returned paper links real rather than plausible-looking fabrications. Output is coerced into `ClaimVerificationResult` and truncated to three papers.

generate_answer_node

Three branches by route. `retrieve` joins all chunks into a context prompt — unless the grader failed after a rewrite, in which case it returns a canned "couldn't find it, try rephrasing" message rather than guessing. `verify_claim` assembles pre-computed state into formatted Markdown with no LLM call at all. `direct_answer` answers from model knowledge.

04 API Surface & Data Flow

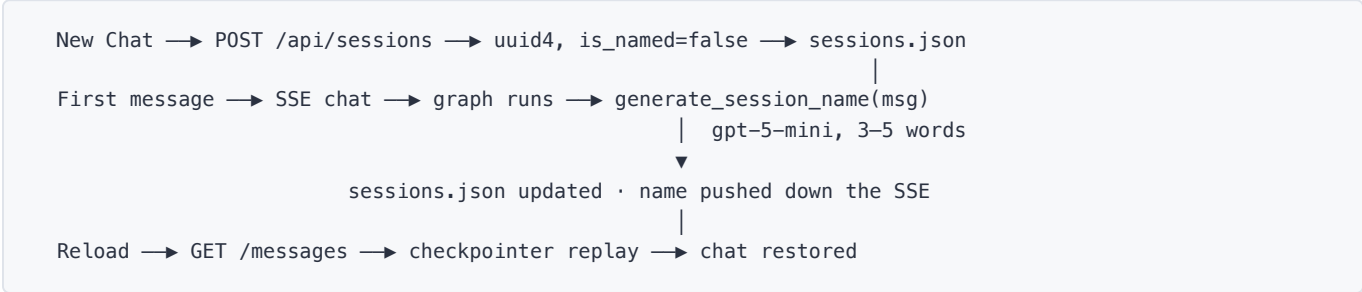
METHOD	PATH	BEHAVIOUR
GET	/api/sessions	All sessions from `sessions.json`, newest first
POST	/api/sessions	Mints a UUID session, `is_named: false` until the first message
GET	/api/sessions/{id}/messages	Replays history out of the LangGraph checkpointer, mapping message classes to chat roles
POST	/api/sessions/{id}/chat	**SSE.** Streams `token` events, then one `done` event carrying the full answer, serialized graph state, and the auto-generated session name
POST	/api/btw	**SSE.** Session-less side channel; nothing is written anywhere
GET	/api/sessions/{id}/documents	Distinct paper titles, gathered by paginated Qdrant scroll over payload metadata
POST	/api/sessions/{id}/documents/files	Multipart upload; per-file result objects so one bad file doesn't fail the batch
POST	/api/sessions/{id}/documents/urls	Batch URL ingestion, same per-item error isolation
POST	/api/sessions/{id}/documents/arxiv	ArXiv ID or title phrase → resolved, downloaded, chunked, embedded

The streaming path

`graph.stream(..., stream_mode="messages")` emits `(chunk, metadata)` pairs for every LLM call in the graph — router, agent, grader, and generator alike. The API filters on `metadata["langgraph_node"] == "generate_answer"`, so the user sees only the final answer forming, not the machinery behind it. Once the stream drains, the API reads the settled state, serializes it (truncating message content and document text to 300 chars to keep the payload small), and emits a terminal `done` event.

Two routes produce no tokens by design — `verify_claim` builds its Markdown deterministically, and the "couldn't find it" fallback is a constant string. Both are covered by the `if not response_text` fallback that pulls `state["answer"]` instead, so those answers arrive in one shot rather than streaming.

Session lifecycle



Two storage systems are in play and they answer different questions. `sessions.json` holds metadata — id, display name, created-at, named-flag — guarded by a `threading.Lock` and rewritten atomically per mutation. The SQLite checkpointer holds everything else, keyed by `thread_id = session_id`. Neither is a cache; a restart restores both.

05 Storage & Retrieval Layer

Vector store

Qdrant Cloud, one collection per session named `papeer_{session_id_with_underscores}`, created lazily on first write with 1536-dimension cosine vectors matching `text-embedding-3-small`. Collection-level namespacing is the isolation mechanism — there is no metadata filtering to get wrong, because one session physically cannot read another's vectors.

Chunking

`RecursiveCharacterTextSplitter` at 1000/200 with `add_start_index=True`, plus a markdown-aware variant built via `from_language("markdown")` that respects heading and list boundaries. Every chunk is stamped with a `title` in metadata, which is what the documents endpoint later scrolls for.

Embedding cache

`CacheBackedEmbeddings` over a `LocalFileStore` at `./embedding_cache/`, namespaced by model name and keyed with blake2b. `query_embedding_cache=True` means repeated *queries* are cached too, not just documents — so re-asking the same question, or re-running the evaluation suite, costs nothing in embedding calls. The working directory currently holds ~120 cached vectors.

Conversation checkpoints

`SqliteSaver` over a connection opened with `check_same_thread=False`, since FastAPI serves requests from a thread pool. Application traffic writes `checkpoints.db`; the evaluation harness is pointed at a separate `eval_checkpoints.db` so test runs never contaminate real sessions.

Paper ingestion by source

SOURCE	LOADER	NOTES
PDF	`PyMuPDFLoader`	Title taken from the filename stem
TXT	`TextLoader (utf-8)`	Standard recursive splitter
Markdown	`TextLoader + md splitter`	Structure-aware splitting on headings and lists
Web URL	`WebBaseLoader`	30 s timeout; page title used, falling back to the URL
ArXiv	`urllib + Atom API`	Regex-extract ID, or title-phrase search; strips version suffix, downloads the PDF to a temp file, parses, deletes

WHY ARXIV IS HAND-ROLLED

The `arxiv` Python package is listed as a dependency but is not used. The loader talks to `export.arxiv.org/api/query` directly over `urllib`, parsing the Atom XML with regex. The README records the reason as reliability problems with the library. The trade-off is a dependency-free, fully predictable path at the cost of brittleness if ArXiv ever changes its Atom shape.

06 Frontend

React 19 with Tailwind 4, built by Vite 8, linted by oxlint. No state library, no router — state lives in `App.tsx` and is threaded down as props.

`App.tsx · 170`

Owns everything: theme (persisted to localStorage, seeded from `prefers-color-scheme`), session list, active session, and a `Record<sessionId, ChatMessage[]>` map so switching sessions is instant and history is fetched lazily on first visit.

`api.ts · 97`

Typed fetch wrappers plus `streamSSE`, an async generator that reads the response body, buffers on `\n\n`, and yields parsed `ChatStreamEvent`s — letting components consume a stream with `for await`.

`ChatPanel.tsx · 147`

Message list, auto-scroll, composer. Detects the `/btw` prefix client-side and routes to the side-channel endpoint.

`DocumentsPanel.tsx · 218`

The largest component. Three ingestion modes with drag-and-drop, per-item success/error feedback, and the loaded-papers list.

`MessageBubble.tsx · 71`

Markdown rendering via `react-markdown` + `remark-gfm`, with a blinking cursor while `pending`.

`GraphStateViewer.tsx · 32`

Collapsible JSON dump of the graph state behind each assistant turn — a genuine debugging affordance exposed in the product UI.

Streaming UX pattern

On send, two messages are appended optimistically — the user's text and an empty assistant bubble marked `pending`. Token events mutate only the last message via `updateLastMessage`, an immutable updater that copies the array and replaces the tail. The `done` event overwrites content with the authoritative answer, attaches `graph_state`, computes the turn number, and clears `pending`. Errors are caught and written into the same bubble, so a failure never leaves a permanently spinning cursor.

07 Evaluation

`evaluate.py` runs a DeepEval harness against `documents/Openclaw_Research_Report.pdf`. Ten goldens were synthesized from that PDF (5 contexts × 2 questions), cached to `goldens.json`, then each replayed through a fresh session and scored on five RAG metrics at a 0.7 threshold. A case counts as *passed* only if all five metrics clear the bar.

4/10	0.99	1.00	0.96	0.59
CASES PASSED	FAITHFULNESS	ANS. RELEVANCY	CTX. PRECISION	CTX. RELEVANCY

Metric breakdown

METRIC	AVG	DISTRIBUTION (THRESHOLD 0.7 MARKED)	PASS RATE	WHAT IT MEASURES
Answer Relevancy	0.997		10 / 10	Does the answer address the question asked
Faithfulness	0.994		10 / 10	Is the answer grounded in retrieved context — no hallucination
Contextual Precision	0.955		9 / 10	Are the most relevant chunks ranked highest
Contextual Recall	0.917		8 / 10	Does retrieval cover everything the expected answer needs
Contextual Relevancy	0.589		4 / 10	What fraction of retrieved text is actually on-topic

Per-case results

#	GOLDEN QUESTION (ABBREVIATED)	CTX. RELEVANCY	RESULT
6	Summarize risks incl. plaintext secrets, auth gaps, public exposure	1.00	Pass
4	Compare local vs. VPS tradeoffs: privacy, latency, uptime, cost	0.97	Pass
9	Strengths and weaknesses justifying the 3.5/5 overall rating	0.80	Pass
5	Delegated-exec tradeoffs: API-key sprawl, async latency	0.72	Pass
1	How native 9+ channels and device-agnostic access improve usability	0.59	Fail
8	Viability for self-hosted power users vs. regulated enterprises	0.57	Fail
10	Plaintext credential storage vs. encrypted secret management	0.44	Fail
7	Niche agentic workflows across shell, browser, email	0.39	Fail
2	Cross-session memory types and how they enable statefulness	0.34	Fail
3	Which LLM backends are supported and how to switch providers	0.07	Fail

READING THE NUMBERS

Every one of the six failures is a contextual-relevancy failure and nothing else. Precision, recall, faithfulness and answer relevancy are all near-ceiling, which says the system finds the right information and reports it honestly — it simply drags a lot of unrelated text along with it. Case 3 at 0.07 is the clearest example: the agent almost certainly retrieved a large `k` for a narrow factual lookup ("which LLM backends are supported"), so the correct passage was buried in noise. Precision stayed high because the right chunk still ranked first.

This is a retrieval-tuning problem, not a generation problem, and it points at three concrete levers: reranking after retrieval, biasing the agent toward smaller `k` for narrow questions, or sentence-level context compression before the answer prompt.

Harness mechanics

- Goldens are generated once and cached; delete `goldens.json` to regenerate.
- Each test case gets a fresh `evaluation_session_{uuid4}` and re-ingests the PDF, so cases cannot leak context into one another.
- Queries are suffixed with *"as per the report in knowledge base"* to force the router down the `retrieve` branch rather than `direct_answer`.
- `AsyncConfig(max_concurrent=3, throttle_value=5)` keeps the five-metric fan-out inside OpenAI rate limits.
- Output lands in `eval_results.json` with per-metric scores, pass flags, and the judge's stated reason for each score.

08 Configuration & Deployment

Required environment

VARIABLE	USED BY	PURPOSE
`OPENAI_API_KEY`	`rag_graph, btw_handler, api, vector_store`	Chat completions and embeddings
`TAVILY_API_KEY`	`rag_graph, btw_handler`	Web search and claim verification
`QDRANT_URL`	`vector_store`	Qdrant Cloud cluster endpoint
`QDRANT_API_KEY`	`vector_store`	Qdrant authentication

Loaded with `python-dotenv` at module import in every module that needs them. `QDRANT_URL` and `QDRANT_API_KEY` are read with bracket access, so a missing key fails loudly at import rather than silently at query time. `.env` is covered by both `.gitignore` and `.dockerignore`.

Runtime

Development

```
uv sync
uv run uvicorn backend.api:app --reload --port 8000

cd frontend && npm install && npm run dev
```

Two processes. Vite serves on 5173 and proxies `/api` to 8000; CORS is explicitly allowed for both localhost spellings of that origin.

Production

```
docker build -t rag-paper-app .
docker run -d -p 8000:8000 \
  -e OPENAI_API_KEY=... -e TAVILY_API_KEY=... \
  -e QDRANT_URL=...     -e QDRANT_API_KEY=... \
  rag-paper-app
```

One process. `api.py` mounts `frontend/dist` at `/` when present, so the API and UI are same-origin and CORS never applies.

Container build

A two-stage Dockerfile. Stage one builds the SPA under `node:22-slim`; stage two runs `python:3.12`, installs from `requirements.txt` (188 pinned packages) first so that layer caches independently of source changes, then copies backend, documents, application files, and finally the built `dist` — ordered least-to-most volatile for cache efficiency. The documented deployment target is EC2 via Docker Hub, with port 8000 opened in the security group.

> **DEPLOYMENT GAP**
>
> The documented `docker run` command mounts no volumes, but `sessions.json` and `checkpoints.db` are written inside the container. Removing the container discards every session and its entire conversation history. Adding `-v papeer-data:/app` — or narrower mounts for those two paths — would make restarts and image upgrades non-destructive. The embedding cache is similarly ephemeral, so a rebuilt container re-pays for embeddings it had already computed.

09 Design Decisions & Trade-offs

DECISION	RATIONALE AND TRADE-OFF
Router-first, retrieval-optional	Costs one extra LLM call per turn, but avoids embedding lookups on questions like "what is softmax" and keeps the answer path honest about which source it used.

Agentic `k` and query control	The agent tunes retrieval per question instead of a fixed `k=4`. More adaptive, but the evaluation suggests it currently over-retrieves — the direct cause of the contextual-relevancy shortfall.
Tools return Command, not text	Document bodies bypass the message history entirely, cutting tokens on every agent turn and keeping the persisted transcript readable.
Rewrite capped at 1, tool calls at 3	Two independent guards against two failure modes — a rewrite loop and a tool-call loop. A hard cap means an unanswerable question degrades to an honest "I couldn't find it" instead of burning tokens indefinitely.
Collection-per-session isolation	Physical separation rather than metadata filtering — impossible to leak across sessions by forgetting a filter. Cost: many small collections and no cross-session search.
1000 / 200 chunking	The 200-char overlap keeps sentences that straddle a boundary retrievable. Larger chunks would raise recall but further depress relevancy.
Two searches for verification	A general search catches blogs and news; the `site:arxiv.org` search catches academic superseding work. Either alone systematically misses one class of evidence.
`/btw` excluded from history	Deliberately out-of-context questions would otherwise distort the model's read of a paper-focused conversation. Cost: no follow-ups within the side channel.
Cached embeddings, singleton graph	Identical text is never re-embedded across sessions or evaluation runs, and the compiled graph is built once at import rather than per request.

10 Observations

Findings from reading the source. None are runtime-breaking; they are the kind of drift that accumulates and is cheapest to fix while the context is fresh.

PRIORITY	FINDING	DETAIL
High	**Model name is inconsistent across modules**	`rag_graph.py:26` and all five metrics in `evaluate.py` instantiate `gpt-5.4-mini`, while `api.py:33`, `btw_handler.py:13`, and the README all use `gpt-5-mini`. Two different models are serving the same product. This should be one constant, read from configuration.
High	**No persistent volume in the deploy path**	As above — `sessions.json` and `checkpoints.db` live inside an ephemeral container. Every redeploy is a full data loss.
Med	**Documentation overstates the rewrite cap**	The README states "Max 3 query rewrites"; `after_relevancy_routing` (`rag_graph.py:383`) permits exactly one. The 3 is `MAX_RETRIEVAL_ATTEMPTS`, which counts tool calls — a different guard entirely.
Med	**Contextual relevancy is the sole blocker to a passing suite**	0.589 average against a 0.7 threshold, and the only reason 6 of 10 cases fail. A reranking step, tighter `k` guidance in the agent prompt, or context compression before the answer prompt would each move this directly.
Med	**Graph state is lost on reload**	`get_session_messages` (`api.py:160`) returns `"graph_state": {}` for every restored turn, so the Graph State Inspector is empty for any conversation loaded from history — it only works for turns generated in the current page session.
Low	**Unused dependencies**	`chromadb` and `arxiv` are declared in `pyproject.toml` but neither is imported anywhere — Qdrant replaced Chroma, and the ArXiv path was hand-rolled. `chromadb` in particular pulls a heavy transitive tree into the image for nothing.
Low	**CORS origins hardcoded**	`api.py:25` allows only `localhost:5173`. Harmless under the single-container deployment where the SPA is same-origin, but any split deployment needs this to become configurable.
Low	**State files are not gitignored**	`sessions.json` and `eval_checkpoints.db` (2.6 MB) sit in the working tree with no ignore rule. `.dockerignore` excludes `*.db`; `.gitignore` does not.
Low	**Constant defined after use**	`MAX_RETRIEVAL_ATTEMPTS` is declared at `rag_graph.py:361` but read at line 194. This resolves correctly at call time, but reads as accidental.
Low	**`main.py` is a stub**	Six lines printing "Hello from rag-papeer-project!". It is copied into the Docker image but is never an entry point — `uvicorn backend.api:app` is.

Suggested sequence

1. **Unify the model name** behind a single `MODEL` constant or env var and pick one value deliberately.
2. **Attach a volume** to the container for `sessions.json`, `checkpoints.db`, and `embedding_cache/`, and update `DOCKER_GUIDE.md` to match.
3. **Attack contextual relevancy** — the fastest experiment is tightening the agent prompt's `k` guidance for narrow questions, then measuring; reranking is the heavier but more durable fix.
4. **Reconcile the README** with the actual rewrite cap, and drop `chromadb` and `arxiv` from `pyproject.toml`.
5. **Persist per-turn graph state** alongside session metadata so the inspector survives a reload.

A Appendix — Repository Inventory

Source

PATH	LOC
backend/rag_graph.py	435
backend/api.py	305
backend/paper_loader.py	117
backend/vector_store.py	88
backend/btw_handler.py	46
backend/models.py	28
evaluate.py	138
main.py	6
frontend/src/components/DocumentsPanel.tsx	218
frontend/src/App.tsx	170
frontend/src/components/ChatPanel.tsx	147
frontend/src/components/icons.tsx	133
frontend/src/components/Sidebar.tsx	133
frontend/src/api.ts	97
frontend/src/components/MessageBubble.tsx	71
frontend/src/components/GraphStateViewer.tsx	32
frontend/src/types.ts	30
Total	**2,205**

Configuration & docs

PATH	ROLE
pyproject.toml	22 direct deps, Python ≥3.12
requirements.txt	188 pinned, for Docker
uv.lock	Resolved lockfile
Dockerfile	Two-stage build
DOCKER_GUIDE.md	Build → Hub → EC2
docker_install_on_AWS.txt	EC2 Docker setup
qdrant_login_steps.md	Cluster provisioning
README.md	Primary documentation
frontend/vite.config.ts	Dev proxy, Tailwind

Runtime artefacts

PATH	CONTENTS
sessions.json	Session metadata
eval_checkpoints.db	2.6 MB eval state
embedding_cache/	~120 cached vectors
goldens.json	10 synthetic Q&A
eval_results.json	Scored run output
graph.png	Rendered graph diagram
documents/	Reference PDF
frontend/dist/	Built SPA assets

Key dependencies

Orchestration

- `langgraph` ≥1.1.10
- `langgraph-checkpoint-sqlite`
- `langchain-core` ≥1.3.3
- `langchain-classic`, `langchain-community`
- `langsmith` ≥0.8.3

Models & retrieval

- `langchain-openai` ≥1.2.1
- `langchain-qdrant`, `qdrant-client`
- `tavily-python` ≥0.7.24
- `pymupdf`, `pypdf`
- `beautifulsoup4`, `lxml`

Serving & testing

- `fastapi` ≥0.120, `uvicorn[standard]`
- `python-multipart`, `python-dotenv`
- `pydantic` ≥2.13.4
- `deepeval` ≥4.0.0
- React 19, Vite 8, Tailwind 4, oxlint

Compiled from static analysis of the working tree at `/Users/shee_sayan/ResearchGPT`. Line counts are raw `wc -l`. Evaluation figures are computed from `eval_results.json` as written by the most recent `evaluate.py` run. No code was executed and no live services were contacted in producing this document.